# WEEK 4: ORDERS AND ORDER DETAILS MANAGEMENT SYSTEM

## Scenario

An e-commerce application needs to maintain information about the orders placed by customers. A customer can place multiple orders, and each order may contain different products.

To manage this process, two new tables are created:

- ORDERS – stores the main information about each order.
- ORDER_DETAILS – stores the products included in each order.

The `ORDERS` table is connected with the existing `CUSTOMER` table. The `ORDER_DETAILS` table connects each order with the products purchased from the existing `PRODUCT` table.

This structure helps the system maintain customer orders and generate useful reports about purchases.

## Design Orders and Order Details Tables

ORDERS TABLE:

```
SQL> CREATE TABLE ORDERS(ORDER_ID VARCHAR2(7), CUSTOMER_ID NUMBER, ORDER_DATE DATE, TOTAL_AMOUNT NUMBER(10,2), ORDER
_STATUS VARCHAR2(20));

Table created.

SQL> DESC ORDERS;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ORDER_ID                                           VARCHAR2(7)
 CUSTOMER_ID                                        NUMBER
 ORDER_DATE                                         DATE
 TOTAL_AMOUNT                                       NUMBER(10,2)
 ORDER_STATUS                                       VARCHAR2(20)
```

ORDER DETAILS TABLE:

```
SQL> CREATE TABLE ORDER_DETAILS(ORDER_DETAIL_ID VARCHAR2(7), ORDER_ID VARCHAR2(7),PRODUCT_ID VARCHAR2(7),QUANTITY NU
MBER, PRICE NUMBER (10,2));

Table created.

SQL> DESC ORDER_DETAILS;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ORDER_DETAIL_ID                                    VARCHAR2(7)
 ORDER_ID                                           VARCHAR2(7)
 PRODUCT_ID                                         VARCHAR2(7)
 QUANTITY                                           NUMBER
 PRICE                                              NUMBER(10,2)
```

# Establish Relationships

The following relationships are established between the tables.

**Customer → Orders**

One customer can place many orders.

**Relationship:**

CUSTOMER (1) ——————————— (M) ORDERS

The `CUSTOMER_ID` in the `ORDERS` table refers to the `CUSTOMER_ID` in the `CUSTOMER` table.

**Orders → Order Details**

One order can contain multiple order details.

ORDERS (1) ——————————— (M) ORDER_DETAILS

**Product → Order Details**

A product can appear in multiple order details.

PRODUCT (1) ——————————— (M) ORDER_DETAILS

Therefore, the many-to-many relationship between customers' orders and products is handled using the `ORDER_DETAILS` table.

**Foreign Key Commands :**

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT PK_ORDERS PRIMARY KEY (ORDER_ID);

Table altered.

SQL> ALTER TABLE ORDERS ADD CONSTRAINT FK_ORDERS_CUSTOMER FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER(CUSTOMER_ID)
;

Table altered.
```

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT PK_ORDER_DETAILS PRIMARY KEY (ORDER_DETAIL_ID);

Table altered.

SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINTS FK_ORDER_DETAILS_ORDER FOREIGN KEY (ORDER_ID) REFERENCES ORDERS(ORDER
_ID);

Table altered.
```

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_ORDER_DETAILS_PRODUCT FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCT(P
RODUCT_ID);

Table altered.
```

## Apply Constraints:

```
SQL> ALTER TABLE ORDERS MODIFY CUSTOMER_ID CONSTRAINT NN_ORDERS_CUSTOMER NOT NULL;

Table altered.

SQL> ALTER TABLE ORDERS MODIFY ORDER_DATE CONSTRAINT NN_ORDERS_DATEE NOT NULL;

Table altered.
```

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_ORDER_QUANTITY CHECK(QUANTITY > 0);

Table altered.

SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_ORDER_PRICE CHECK(PRICE > 0);

Table altered.

SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_ORDER_AMOUNT CHECK(TOTAL_AMOUNT > 0);

Table altered.

SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_ORDER_STATUS CHECK(ORDER_STATUS IN ('PENDING','SHIPPED','DELIVERED'));

Table altered.
```

## Insert Values :

ORDER table

```
SQL> INSERT INTO ORDERS VALUES('&ORDER_ID',&CUSTOMER_ID,DATE '&ORDER_DATE',&TOTAL_AMOUNT,'&ORDER_STATUS');
Enter value for order_id: O001
Enter value for customer_id: 1
Enter value for order_date: 2026-09-01
Enter value for total_amount: 2500.00
Enter value for order_status: DELIVERED
old   1: INSERT INTO ORDERS VALUES('&ORDER_ID',&CUSTOMER_ID,DATE '&ORDER_DATE',&TOTAL_AMOUNT,'&ORDER_STATUS')
new   1: INSERT INTO ORDERS VALUES('O001',1,DATE '2026-09-01',2500.00,'DELIVERED')

1 row created.
```

```
SQL> SELECT * FROM ORDERS;

ORDER_I CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
------- ----------- --------- ------------ --------------------
O001              1 01-SEP-26         2500 DELIVERED
O002              2 02-SEP-26         1800 SHIPPED
O003              1 03-SEP-26         3200 DELIVERED
O004              3 04-SEP-26         1500 PENDING
O005              4 05-SEP-26         4200 DELIVERED
O006              2 06-SEP-26         2750 SHIPPED
O007              5 07-SEP-26         3600 DELIVERED
O008              6 08-SEP-26         1200 PENDING
O009              1 09-SEP-26         5100 DELIVERED
O010              7 10-SEP-26         2300 SHIPPED
O011              8 11-SEP-26         1900 DELIVERED
O012              3 12-SEP-26         3400 PENDING
O013              9 13-SEP-26         2800 SHIPPED
O014             10 14-SEP-26         4500 DELIVERED
O015              4 15-SEP-26         1600 PENDING

15 rows selected.
```

ORDER DETAILS table:

```
SQL> INSERT INTO ORDER_DETAILS VALUES('&ORDER_DETAIL_ID','&ORDER_ID','&PRODUCT_ID',&QUANTITY,&PRICE);
Enter value for order_detail_id: OD001
Enter value for order_id: O001
Enter value for product_id: P001
Enter value for quantity: 1
Enter value for price: 2500
old   1: INSERT INTO ORDER_DETAILS VALUES('&ORDER_DETAIL_ID','&ORDER_ID','&PRODUCT_ID',&QUANTITY,&PRICE)
new   1: INSERT INTO ORDER_DETAILS VALUES('OD001','O001','P001',1,2500)

1 row created.
```

```
SQL> SELECT * FROM ORDER_DETAILS;

ORDER_D ORDER_I PRODUCT   QUANTITY      PRICE
------- ------- ------- ---------- ----------
OD001   O001    P001             1       2500
OD002   O002    P002             2        900
OD003   O003    P003             1       3200
OD004   O004    P004             3        500
OD005   O005    P005             2       2100
OD006   O006    P006             1       2750
OD007   O007    P007             2       1800
OD008   O008    P008             1       1200
OD009   O009    P009             3       1700
OD010   O010    P010             1       2300
OD011   O011    P011             2        950
OD012   O012    P012             1       3400
OD013   O013    P013             1       4500
OD014   O014    P014             1       4500
OD015   O015    P015             2        800

15 rows selected.
```

## Report 1: Customer Order History

This report displays the order history of customers. It combines customer information with their order details.

### Description

The query joins the `CUSTOMER` and `ORDERS` tables using `CUSTOMER_ID`.

It displays:

- Customer name
- Order ID
- Order date
- Total amount
- Order status

This report helps in viewing the complete order history of each customer.

```
SQL> SELECT c.CUSTOMER_NAME,
  2         o.ORDER_ID,
  3         o.ORDER_DATE,
  4         o.TOTAL_AMOUNT,
  5         o.ORDER_STATUS
  6  FROM CUSTOMER c
  7  JOIN ORDERS o
  8  ON c.CUSTOMER_ID = o.CUSTOMER_ID
  9  ORDER BY c.CUSTOMER_NAME, o.ORDER_DATE;

CUSTOMER_NAME                                      ORDER_I ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
-------------------------------------------------- ------- --------- ------------ --------------------
Ajay Kumar                                         O011    11-SEP-26         1900 DELIVERED
Anjali Devi                                        O007    07-SEP-26         3600 DELIVERED
Arun Kumar                                         O001    01-SEP-26         2500 DELIVERED
Arun Kumar                                         O003    03-SEP-26         3200 DELIVERED
Arun Kumar                                         O009    09-SEP-26         5100 DELIVERED
Karthik M                                          O005    05-SEP-26         4200 DELIVERED
Karthik M                                          O015    15-SEP-26         1600 PENDING
Manoj R                                            O013    13-SEP-26         2800 SHIPPED
Meeena K                                           O010    10-SEP-26         2300 SHIPPED
Nithya S                                           O014    14-SEP-26         4500 DELIVERED
Priya Sharma                                       O002    02-SEP-26         1800 SHIPPED
Priya Sharma                                       O006    06-SEP-26         2750 SHIPPED
Rahul Raj                                          O004    04-SEP-26         1500 PENDING
Rahul Raj                                          O012    12-SEP-26         3400 PENDING
Vijay Kumar                                        O008    08-SEP-26         1200 PENDING

15 rows selected.
```

**Report 2: Product-wise Order Report**
This report provides information about the number of orders and total quantity ordered for each product.

Description

The query connects the `PRODUCT` and `ORDER_DETAILS` tables.

It calculates:

- Product name
- Number of orders
- Total quantity ordered

The `COUNT()` function is used to count orders, while `SUM()` calculates the total quantity.

The result is arranged according to the total quantity ordered.

```
SQL> SELECT p.PRODUCT_NAME,
  2         COUNT(od.ORDER_ID) AS ORDER_COUNT,
  3         SUM(od.QUANTITY) AS TOTAL_QUANTITY
  4  FROM PRODUCT p
  5  JOIN ORDER_DETAILS od
  6  ON p.PRODUCT_ID = od.PRODUCT_ID
  7  GROUP BY p.PRODUCT_NAME
  8  ORDER BY TOTAL_QUANTITY DESC;

PRODUCT_NAME         ORDER_COUNT TOTAL_QUANTITY
-------------------- ----------- --------------
Running Shoes                  1              3
Cotton Shirt                   1              3
Yoga Mat                       1              2
Building Blocks                1              2
Python programming             1              2
Denim Jeans                    1              2
Wireless Mouse                 1              2
Study Table                    1              1
Electric Kettle                1              1
Bluetooth Speaker              1              1
Data Structures                1              1
Office Chair                   1              1
Cooking oil                    1              1
Face wash                      1              1
Smart phone                    1              1

15 rows selected.
```

### Report 3: Customer Purchase Analysis

This report analyzes the purchasing activity of customers.

It shows the total number of orders, total amount spent and average order value for each customer.

#### Description

The query uses aggregate functions to analyze customer purchases.

- `COUNT()` calculates the total number of orders.
- `SUM()` calculates the total spending.
- `AVG()` calculates the average order value.
- `ROUND()` displays the average value up to two decimal places.

The report helps to understand the purchasing pattern of customers.

```
SQL> SELECT c.CUSTOMER_NAME,
  2         COUNT(o.ORDER_ID) AS TOTAL_ORDERS,
  3         SUM(o.TOTAL_AMOUNT) AS TOTAL_SPENDING,
  4         ROUND(AVG(o.TOTAL_AMOUNT), 2) AS AVERAGE_ORDER_VALUE
  5  FROM CUSTOMER c
  6  JOIN ORDERS o
  7  ON c.CUSTOMER_ID = o.CUSTOMER_ID
  8  GROUP BY c.CUSTOMER_NAME
  9  ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_NAME                                      TOTAL_ORDERS TOTAL_SPENDING AVERAGE_ORDER_VALUE
-------------------------------------------------- ------------ -------------- -------------------
Arun Kumar                                                    3          10800                3600
Karthik M                                                     2           5800                2900
Rahul Raj                                                     2           4900                2450
Priya Sharma                                                  2           4550                2275
Nithya S                                                      1           4500                4500
Anjali Devi                                                   1           3600                3600
Manoj R                                                       1           2800                2800
Meeena K                                                      1           2300                2300
Ajay Kumar                                                    1           1900                1900
Vijay Kumar                                                   1           1200                1200

10 rows selected.
```

# Conclusion

In this week, the ORDERS and ORDER_DETAILS tables were created for the e-commerce database.The required primary key, foreign key, NOT NULL and CHECK constraints were applied to maintain data accuracy. The relationships between customers, orders and products were also established.These reports help to understand customer orders, product demand and customer purchasing activity. Thus, the order management part of the e-commerce database was successfully implemented.